

Credit Card Fraud Detection using Machine Learning

Project Description

This project automates the detection of fraudulent credit card transactions using machine learning. Banks process thousands of transactions per second, and fraudulent ones make up less than 0.2% of them – far too rare and subtle for manual, rule-based review to catch reliably at scale. The system is trained on the ULB Credit Card Fraud Detection dataset (284,807 anonymized European transactions from September 2013) and evaluates four classification algorithms – Logistic Regression, Random Forest, XGBoost, and Decision Tree – to identify the model that best separates genuine from fraudulent activity, even under extreme class imbalance.

The best-performing model is saved and served through a Flask web application, allowing an analyst to submit a transaction’s features and instantly receive a Fraud/Genuine prediction with a confidence score. An IBM Watson Machine Learning deployment pipeline is also included so the model can be hosted on the cloud for scalable, real-time scoring.

Scenarios

Scenario 1: Real-Time Transaction Screening. A transaction is submitted for scoring the moment it occurs. The Flask API returns a Fraud/Genuine prediction with a probability score in under a second, allowing automated systems to approve, decline, or flag the transaction for review.

Scenario 2: High-Risk Case Triage. A fraud analyst reviews the day’s flagged transactions on the dashboard, sorted by predicted fraud probability, so the highest-risk cases get attention first instead of reviewing every transaction manually.

Scenario 3: Compliance Reporting. A compliance officer exports prediction logs to review the model’s decisions and confirm they align with actual chargeback/dispute outcomes over time.

Technical Architecture

The system uses a three-layer architecture: a Flask-based presentation layer (HTML form for single-transaction input), a core logic layer (feature scaling + model inference using scikit-learn/XGBoost), and a data layer (the saved model artifact and prediction logs). For cloud-scale deployment, the trained model is packaged and pushed to IBM Watson Machine Learning, which exposes a hosted scoring endpoint that the Flask app (or any other client) can call instead of running inference locally.

(Note: No relational database is used in the current scope since no persistent user accounts are required – this project does not need an ER diagram.)

Pre-requisites

- Python 3.10+ and pip
- Flask framework knowledge
- scikit-learn, XGBoost, and imbalanced-learn (SMOTE) familiarity
- An IBM Cloud account with Watson Machine Learning service (for cloud deployment)
- Git for version control

Project Workflow

Milestone 1: Data Understanding & Preprocessing - Activity 1.1: Load the `creditcard.csv` dataset and inspect class distribution (492 frauds out of 284,807 transactions). - Activity 1.2: Scale the Time and Amount features with `StandardScaler` (the `v1-v28` features are already PCA-standardized). - Activity 1.3: Split into train/test sets using stratified sampling to preserve the fraud ratio in both.

Milestone 2: Handling Class Imbalance - Activity 2.1: Apply SMOTE to the training set only, so evaluation on the untouched test set reflects real-world class proportions. - Activity 2.2: Confirm the resampled training set has a balanced fraud/genuine ratio before model training.

Milestone 3: Model Training & Comparison - Activity 3.1: Train Logistic Regression, Decision Tree, Random Forest, and XGBoost classifiers on the resampled training data. - Activity 3.2: Evaluate each on the held-out test set using Precision, Recall, F1-score, and ROC-AUC – prioritizing Recall and F1 over raw Accuracy, since accuracy is misleading on a 0.17%-positive-class dataset. - Activity 3.3: Select the best-performing model and save it (`best_model.pkl`) along with the fitted scaler.

Milestone 4: Flask Web Application - Activity 4.1: Build a form (`index.html`) where a user enters transaction feature values (Time, Amount, `v1-v28`). - Activity 4.2: Implement the `/predict` route in `app.py`, which scales the input, runs the saved model, and returns a JSON response with the prediction and fraud probability.

Milestone 5: Cloud Deployment (IBM Watson ML) - Activity 5.1: Package the trained model and push it to the IBM Watson Machine Learning repository using `deploy_watson_ml.py`. - Activity 5.2: Create an online deployment so the model

can be scored via a hosted REST endpoint instead of running only on a local machine.

Milestone 6: Testing & Conclusion - Activity 6.1: Performance-test the `/predict` endpoint under simulated concurrent load (see Project Testing phase). - Activity 6.2: Document setup instructions, known limitations, and a future scalability roadmap.

Conclusion

This project demonstrates that a supervised machine learning pipeline – with careful handling of extreme class imbalance via SMOTE – can reliably separate fraudulent from genuine credit card transactions using only anonymized, PCA-transformed features. XGBoost and Random Forest in particular capture non-linear fraud patterns that a simple rule-based system would miss. Wrapping the best model in a lightweight Flask API, with a path to IBM Watson ML for scalable hosting, shows a practical route from a trained model to a usable, real-time fraud-screening tool for a bank’s risk team.